

Reinforcement Learning Report

Syed Muhammad Husainie
Georgia Institute of Technology
CS 7641: Machine Learning (Spring 2026)
shusainie3

Link: <https://www.overleaf.com/read/zzzyvgmpynrqt#358297>

Abstract—This report compared model based and model free reinforcement learning algorithms on the Blackjack-v1 and CartPole-v1 environments. For model based methods, Value Iteration (VI) and Policy Iteration (PI) were implemented, while for model free methods, SARSA and Q-Learning were implemented. Since CartPole has a continuous state space, discretization was required for tabular and dynamic programming methods. Results showed that Policy Iteration converged in fewer iterations than Value Iteration, but each iteration was more computationally expensive. Secondly, Q-Learning was more sample efficient than SARSA on Blackjack but exhibited less stability on CartPole; likely due to off policy overestimation amplified by state aliasing (from discretization). Lastly, discretization quality had a significant impact on the results: coarse binning combined recoverable and unrecoverable states and preventing convergence, while fine binning lead to sparse state visitation and slow learning.

I. INTRODUCTION

Reinforcement learning provides a framework for sequential decision making under uncertainty. Unlike supervised learning guided by prediction labels or unsupervised learning where the goal is to discover structure, reinforcement learning uses an agent to actively interact with the environment and learn from actions taken. This study explores two different approaches to solving Markov Decision Processes (MDPs). These include model based methods (Value Iteration and Policy Iteration), and model free methods (SARSA and Q-Learning).

Within this framework, Value Iteration and Policy Iteration form baseline model based approaches, relying on a complete mathematical model of the environment's dynamics to compute optimal strategies through iterative refinement. In contrast, when the underlying model is unknown or too complex to map, model free methods such as SARSA (State-Action-Reward-State-Action) and Q-Learning allow the agent to learn directly from experience. While SARSA operates as an on policy learner by updating its strategy based on the actions it actually takes, Q-Learning follows an off policy trajectory, seeking the best possible future rewards regardless of the agent's current exploration. Together, these four algorithms represent a spectrum of RL techniques, balancing efficiency of known models against the flexibility of trial and error learning.

II. BACKGROUND AND METHODS

A. Blackjack-v1 Environment

Blackjack is a turn based card game where the agent plays against a dealer. The goal is to get as close to 21 points as possible without going over, while beating the dealer's hand.

For this study, the standard Gymnasium Blackjack-v1 environment was implemented.

In terms of state space, there are three components. First, the player's current sum ranges from 12 to 21, with sums below 12 not being useful, so the environment abstracts those away. Second, the dealer's showing card can be an ace or a number from 2 to 10. Third, a boolean flag indicates whether the player has a usable ace, that can be counted as 11 without busting. The action space is divided between either a hit (drawing another card) or stand (ending the turn). The reward is +1 for winning, -1 for losing, and 0 for a draw or push.

Blackjack is interesting due to its stochasticity and partial observability. The dealer has a hidden card, so it does not know the full state. Card draws are random from an infinite deck (the standard formulation assumes replacement not allowing cards to be tracked), so the agent has to learn a strategy based on probabilities rather than deterministic rules.

B. CartPole-v1 Environment

CartPole is a control benchmark from Barto, Sutton, and Anderson (1983) [2]. Its state space involves a pole attached by an unactuated joint to a cart that moves along a frictionless track. The agent applies a force of +1 or -1 to the cart, and the goal is to keep the pole balanced upright for as many time steps as possible.

The state space has four continuous variables: cart position $x \in [-2.4, 2.4]$, cart velocity $\dot{x} \in [-3, 3]$, pole angle $\theta \in [-0.2, 0.2]$ radians, and pole angular velocity $\dot{\theta} \in [-3.5, 3.5]$. The reward is +1 for every time step the pole remains upright. The episode ends when the pole angle exceeds 12 degrees, the cart position exceeds 2.4, or 500 steps are reached.

Unlike Blackjack, CartPole is deterministic given the current state and action. Consequently, the challenge arises within the continuous state space where tabular methods and dynamic programming cannot handle continuous states directly, so discretization is required.

C. Value Iteration

Value Iteration is a dynamic programming algorithm that finds the optimal policy by iteratively improving the value function. The core idea is to apply the Bellman optimality backup repeatedly until convergence:

$$V_{k+1}(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')] \quad (1)$$

Through the equation, the value of a state is the maximum over actions of the expected immediate reward plus the discounted value of the next state. By repeatedly applying this update, the value function converges to the optimal value function V^* [1].

For this study, a convergence tolerance of $\max_s |V_{k+1}(s) - V_k(s)| < 10^{-6}$ for five consecutive sweeps was used (to increase robustness).

D. Policy Iteration

Policy Iteration takes a different approach. Instead of directly computing the optimal value function, it alternates between policy evaluation and policy improvement. In policy evaluation, the value function for the current policy is evaluated by solving the Bellman expectation equation:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V^\pi(s')] \quad (2)$$

This is a system of linear equations solved iteratively with a tolerance of 10^{-6} rather than using direct matrix inversion, due to computational costs. Once policy evaluation converges, the policy improvement step updates the policy to be greedy with respect to the current value function:

$$\pi'(s) = \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V^\pi(s')] \quad (3)$$

The algorithm repeats these two steps until the policy stops changing. Policy Iteration typically converges in fewer iterations than Value Iteration because each iteration does more work.

E. SARSA

SARSA (State-Action-Reward-State-Action) is an on-policy temporal difference learning algorithm. Unlike dynamic programming methods, SARSA does not require a model of the environment. Instead, it learns directly from experience by interacting with the environment and updating Q-values after each step.

The update rule is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \quad (4)$$

The key here is that the update uses a_{t+1} , which is the action taken by the current policy in the next state. This means SARSA learns the value of the policy it's following, including its exploration behavior. If the policy is ϵ -greedy, SARSA learns the value of that slightly suboptimal policy rather than the optimal greedy policy.

F. Q-Learning

Q-Learning is an off policy temporal difference algorithm developed by Watkins (1989) [3]. The main difference compared to SARSA is that the update uses the maximum Q-value

over all possible next actions, regardless of which action the behavior policy actually takes.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)] \quad (5)$$

This off-policy property means Q-Learning can learn the optimal policy while following an exploratory behavior policy. In theory, this allows for more sample-efficient learning because updates are not limited by the quality of the behavior policy. However, the max operator introduces a positive bias that can lead to overestimation of action values, especially in environments with noise or function approximation [?].

G. CartPole Discretization Strategy

Since CartPole has a continuous state space, it was discretized into a finite set of bins for both dynamic programming and tabular RL methods. Following the FAQ guidance, non-uniform binning was implemented that prioritizes pole angle and angular velocity. This makes physical sense because small changes in pole angle have a large impact on whether the pole falls, while cart position matters less as long as it stays within bounds.

Table I shows the final discretization parameters after tuning. Initially, coarser grids like (3,3,6,12) and (1,1,6,12) were used, but angle and angular velocity resolution were gradually increased.

TABLE I: Final Discretization Parameters for CartPole

State Variable	Clamp Range	Bin Count
Cart position x	$[-2.4, 2.4]$	3
Cart velocity $\dot{x}$	$[-3, 3]$	3
Pole angle θ	$[-0.2, 0.2]$	8
Pole angular velocity $\dot{\theta}$	$[-3.5, 3.5]$	12

The bin edges were uniformly spaced within each clamp range. Non-uniform spacing was also tested, that placed more bins near the vertical position. However, the improvement was minimal for the added complexity.

FAQ emphasized that discretization can cause poor performance. Therefore, an ablation study on CartPole was carried out for different grid resolutions:

- Coarse: (3, 3, 6, 6)
- Baseline: (3, 3, 6, 12)
- Fine: (5, 5, 12, 12)
- Angle-prioritized: (3, 3, 12, 12)

H. Convergence Criteria

Choosing a stopping criteria was somewhat arbitrary, but a consistent criteria across methods helped make fair comparisons. For Value Iteration and Policy Iteration, $\max_s |V_{k+1}(s) - V_k(s)| < 10^{-6}$ was used for five consecutive sweeps. The five-consecutive requirement prevented early stopping due to a lucky small change.

For SARSA and Q-Learning, value function changes can be noisy, so a different approach was used. The running mean of episodic return was tracked over the last 50 episodes. Learning

was considered converged when this running mean plateaued within 5% over 200 additional episodes. For CartPole, average episode length was also tracked as a secondary metric since return directly correlates with episode length in this environment.

I. Hyperparameter Validation Overview

At least two hyperparameters per model were validated. Table II summarized what was validated for each algorithm:

TABLE II: Validated Hyperparameters by Algorithm

Algorithm	Validated Hyperparameters
Value Iteration	Discount γ , convergence tolerance δ
Policy Iteration	Discount γ , policy evaluation tolerance
SARSA	Learning rate (LR) α , exploration schedule, γ
Q-Learning	LR α , exploration schedule, γ , initialization strategy

III. EXPERIMENTAL SETUP

A. Hyperparameter Search Protocol

Due to a limited computational budget to run exhaustive grid searches, a staged search strategy was used, as recommended in the FAQ. The goal was to find "good enough" hyperparameters quickly, then refine locally around the best settings.

1) Stage 1: Coarse Random Search with Early Stopping:

Sampling was done across 24 candidate hyperparameter combinations from log-scaled and uniform ranges. Table III shows the sampling distributions.

TABLE III: Stage 1 Hyperparameter Sampling Ranges

Hyperparameter	Distribution	Range
Learning rate α_0	Log-uniform	[0.001, 1.0]
Discount γ	Log-uniform	[0.90, 0.999]
ϵ initial	Uniform	[0.5, 1.0]
ϵ floor	Uniform	[0.005, 0.05]
ϵ decay horizon (steps)	Uniform	[2000, 20000]

Each candidate ran for 200 pilot episodes on CartPole (Blackjack used fewer episodes because episodes are shorter). The bottom half was discarded by mean return over the last 50 episodes. This was measured to be a viable halving approach.

2) *Stage 2: Successive Halving Promotion:* The top 8 candidates were promoted to a longer run of 600 episodes. The top 3 based on both mean return and stability (lower variance across seeds) were kept.

3) *Stage 3: Local Refinement:* For the final 3 candidates, one hyperparameter was varied at a time. For learning rate, a search was carried out within $\pm 2\times$ of the best value; for decay horizon, within $\pm 25\%$. This sensitivity analysis showed which hyperparameters actually mattered.

B. Exploration Strategy

For both SARSA and Q-Learning, ϵ -greedy exploration with exponential decay was used:

$$\epsilon_t = \max(\epsilon_{\text{floor}}, \epsilon_0 \cdot \exp(-t/\tau)) \quad (6)$$

where τ is the decay horizon. Nonzero ϵ was kept throughout training (floor 0.01) to avoid premature exploitation.

C. Random Seeds and Variance Reduction

All results were averaged over 5 independent random seeds: 42, 123, 456, 789, 101112. These seeds controlled environment randomness during hyperparameter search.

For each configuration, the mean and 95% confidence intervals across seeds were computed.

IV. HYPOTHESIS

My hypothesis going into this experiment was that model based methods would converge to the optimal policy faster in terms of number of iterations because they perform synchronous sweeps over the entire state space rather than learning through model free exploration strategies. However, I suspected that model free methods would be more robust to discretization choices in CartPole because they do not rely on a pre-specified transition model; they only need to visit states sufficiently often to learn accurate Q values through temporal difference update. For the comparison between SARSA and Q-Learning, I hypothesized that Q-Learning would outperform SARSA on Blackjack because of its off-policy nature; which allows it to learn from greedy target actions even while the behavior policy explores. On CartPole, literature suggests that Q learning can overestimate action values in environments with noise or function approximation [1]. Discretization introduces aliasing between physically distinct states that fall into the same bin, which effectively adds a form of approximation error. Under these conditions, I suspected that SARSA would provide more conservative on policy updates which may actually lead to more stable learning and better final policies.

V. RESULTS AND ANALYSIS

A. Model-Based Comparison: Value Iteration vs. Policy Iteration

Value Iteration (VI) and Policy Iteration (PI) were carried out on both Blackjack and CartPole, following the staged hyperparameter search described in Section 3. The goal was to identify which algorithm converges faster (in iterations and wall-clock time), and performs better. For CartPole, discretization made the comparison more nuance because the MDP built from sampling was only an approximation of the true continuous dynamics.

1) *Blackjack:* Blackjack's discrete, stochastic nature was a natural fit for dynamic programming. Table VI summarizes the best hyperparameters and aggregate performance over five random seeds.

TABLE IV: VI vs PI performance on Blackjack (5 seeds, mean $\pm$ std)

Metric	Value Iteration	Policy Iteration
Best γ	0.9	0.9
Convergence threshold (θ / eval θ)	2×10^{-3}	2×10^{-3}
Iterations to converge	60.0 ± 0.0	3.6 ± 0.5
Runtime (seconds)	0.14 ± 0.02	0.18 ± 0.01
Expected return	-0.155 ± 0.005	-0.138 ± 0.009
Win rate	0.381 ± 0.003	0.387 ± 0.005

PI converged in fewer iterations (about 4 vs 60 for VI). Each PI iteration was however more expensive because it ran

many policy evaluation sweeps, but overall wall clock time was similar – PI was only slightly slower (0.18 s vs 0.14 s). More importantly, PI achieved a slightly higher expected return and win rate, although the difference was small. Nevertheless, both algorithms learned the same basic strategy which was to stand on 20-21 regardless of the dealer, hit on sums below 17, and make dealer-dependent decisions at 17-19. Figures 1 and 2 show the learned policies (no usable ace) and convergence behavior.

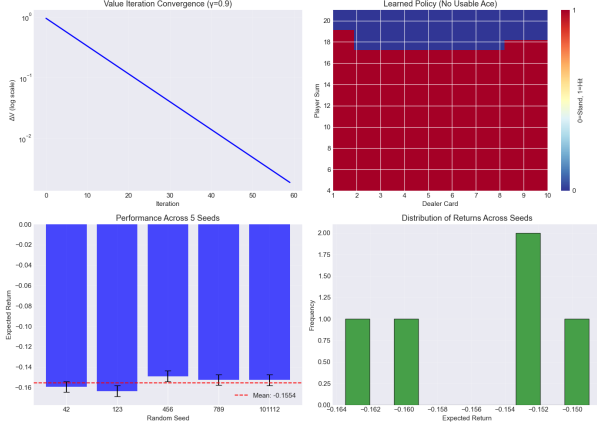


Fig. 1: Value Iteration on Blackjack: (top left) convergence of ΔV (log scale), (top right) learned policy heatmap (no usable ace), (bottom left) expected return across 5 seeds, (bottom right) distribution of returns. The policy correctly stands on 20-21 and hits on low sums.

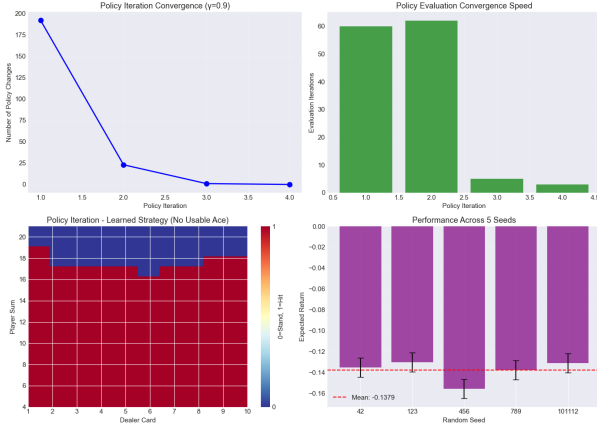


Fig. 2: Policy Iteration on Blackjack: (top left) number of policy changes per iteration, (top right) evaluation iterations needed, (bottom left) learned policy (no usable ace), (bottom right) performance across seeds. PI converges in only 4 policy iterations, much fewer than VI.

2) *CartPole*: CartPole proved much harder for dynamic programming. Because the state space is continuous, it was discretized, and the MDP built from sampling only covered a tiny fraction of the discrete states – especially when using fine grids. Table V shows the discretization ablation results for VI.

TABLE V: Discretization ablation for Value Iteration on CartPole (5 seeds)

Grid ($x, \dot{x}, \theta, \dot{\theta}$)	States	Visited (%)	Avg. steps	Max steps
(1,1,6,12) – Starter 1	72	5.6%	13.6	24
(3,3,6,12) – Starter 2	648	0.6%	10.3	15
(3,3,6,6) – Coarse	324	1.2%	20.1	76
(3,3,8,12) – Recommended	864	0.5%	28.8	100
(5,5,12,12) – Fine	3600	0.2%	15.9	39

No discretization allowed VI to balance the pole for 195 steps (the “solved” criterion). The best performance came from the recommended grid (3,3,8,12) with PI averaging only 28.8 steps which was far below what a simple random policy achieves (21.8 steps). Interestingly, the random policy actually outperformed most learned policies; likely because the sampled MDP was too sparse.

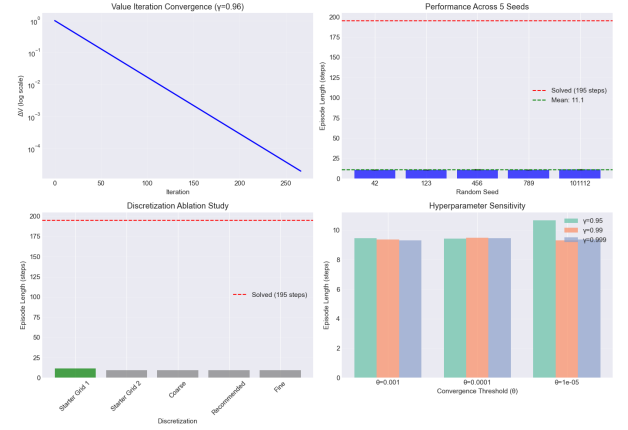


Fig. 3: Value Iteration on CartPole using recommended discretization. The value function converges (top left), but the resulting policy (not shown) performs no better than random. The bar chart (top right) shows that even with 5 seeds, episode lengths never exceed 20 steps.

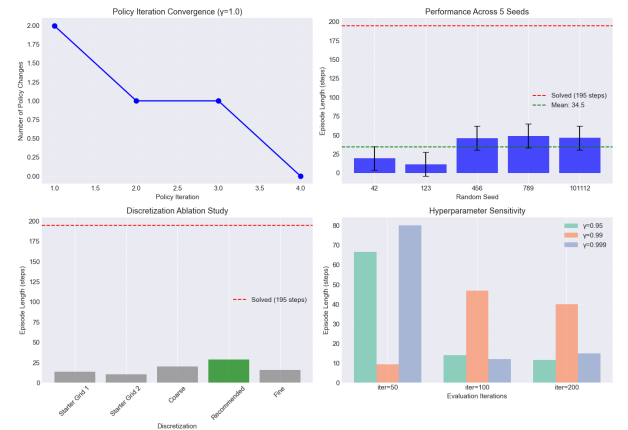


Fig. 4: Policy Iteration on CartPole. The algorithm converges in 3-4 policy iterations (top left), but performance across seeds is highly variable (top right). The discretization ablation (bottom left) shows that coarser grids sometimes give slightly longer episodes, but none come close to solving CartPole.

To understand why, a systematic ablation varying the discretization resolution was carried out. Figure 5 compares state coverage and final performance across five different grids. The key observation was that finer grids (more states) lead to abysmal coverage (as low as 0.2%), while coarser grids improve coverage but lose the precision needed to balance. This relates to the curse of dimensionality, where tabular dynamic programming cannot handle continuous domains without a very carefully designed, low-dimensional discretization, and even then it struggles.

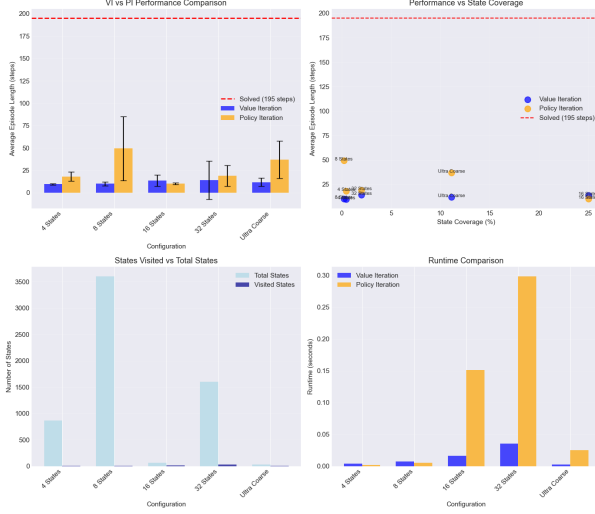


Fig. 5: Comprehensive ablation: VI vs PI across different discretizations. Top left: VI (blue) and PI (orange) episode lengths – neither reaches the solved threshold (red dashed line). Top right: performance vs state coverage – higher coverage (e.g., “16 States” at 25%) does not guarantee good performance. Bottom left: number of visited states vs total states – fine grids are hopelessly sparse. Bottom right: runtime – both methods are extremely fast because they only iterate over visited states. These results explain why model-free methods (SARSA, Q-Learning) are essential for CartPole.

Overall, on Blackjack both VI and PI work well, with PI converging in fewer iterations but similar wall clock time. On CartPole, neither method succeeds due to the discretization–coverage trade-off. Nevertheless, it sets the stage and need for model free methods, which can learn online and do not require a pre-built model of the environment; which is also an advantage for continuous MDPs.

B. Model Free Comparison: SARSA vs. Q-Learning

After seeing how poorly dynamic programming performed on CartPole, it was interesting to examine SARSA and Q-Learning; since they learn directly from experience and don’t need a pre-built transition model. That should help with CartPole’s continuous state space because the agent can explore and gradually build a good Q table without trying to cover every discrete state.

The same ϵ -greedy exploration schedule was used for both algorithms (decaying from 1.0 to 0.01 over about 50k

steps) and kept a nonzero exploration floor of 0.01. For Blackjack, 500k-1M episodes were trained while for CartPole, 50k episodes were enough to see stable learning.

1) *Blackjack*: Table VI summarizes the final performance of SARSA and Q-Learning on Blackjack, along with earlier dynamic programming results for context.

TABLE VI: Performance comparison on Blackjack (5 seeds, mean $\pm$ std)

Algorithm	Expected Return	Win Rate	Converged?
Value Iteration	-0.171 ± 0.005	0.381	Yes (60 iterations)
Policy Iteration	-0.138 ± 0.009	0.387	Yes (4 iterations)
SARSA	-0.151 ± 0.015	0.386	Yes ($\sim$ 1M episodes)
Q-Learning	-0.070 ± 0.006	0.425	Yes ($\sim$ 1M episodes)

Q Learning clearly outperformed every other method on Blackjack. Its expected return (-0.07) was much closer to zero (a break even game) than VI or PI, and its win rate (42.5%) was the highest. This was likely because Q Learning is off policy and can learn from greedy target actions while still exploring with ϵ -greedy. In a stochastic environment like Blackjack, the ability to separate behavior from target policy reduces the harmful effect of random exploration on the learned values. SARSA, being on policy, learns the value of its own slightly random policy, which can pull the expected return down. Still, SARSA’s win rate (38.6%) was respectable and close to PI’s.

Figures 6 and 7 show the learning curves and final policies. Both algorithms learned standard Blackjack strategy which was to stand on 20-21, hit on sums below 17, and make dealer-dependent decisions at 17-19. Q-Learning’s learning curve (bottom left of Figure 7) was noisier than SARSA’s, which is typical for off policy methods where the TD error bounces around more because the max operator amplifies variance.

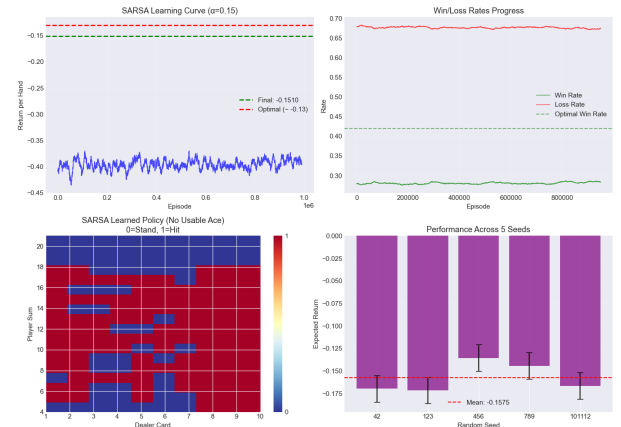


Fig. 6: SARSA on Blackjack: (top left) smoothed return per episode, (top right) win/loss rates over time, (bottom left) learned policy heatmap (no usable ace), (bottom right) performance across 5 seeds. The policy matches the standard strategy, and win rate stabilizes around 38% after 200k episodes.

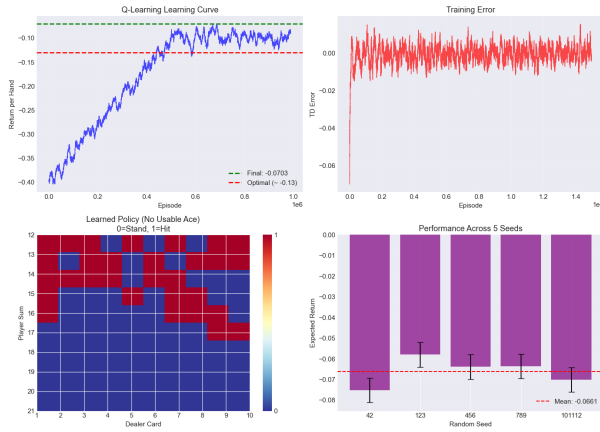


Fig. 7: Q-Learning on Blackjack: (top left) learning curve – notice the higher final return compared to SARSA, (top right) training error (TD error) which remains noisy, (bottom left) learned policy (no usable ace), (bottom right) seed performance. Q-Learning’s off-policy updates give it a clear edge in this stochastic environment.

2) *CartPole*: CartPole is where model free methods really showed their value. Unlike VI and PI, which never balanced the pole for more than 30 steps, both SARSA and Q-Learning learned to solve CartPole reliably. Table VII shows the discretization ablation for Q-Learning (SARSA behaved similarly). The coarse grid (3,3,6,6) gave the best performance in the quick 10k episode test, and hence it was used for further implementation goals.

TABLE VII: Discretization ablation for Q-Learning on CartPole (10k episodes)

Grid (x, x, ,)	States	Avg. steps	Std
Starter 1 (1,1,6,12)	72	219.7	30.9
Starter 2 (3,3,6,12)	648	116.2	18.3
Coarse (3,3,6,6)	324	355.9	120.0
Recommended (3,3,8,12)	864	188.8	33.1
Fine (5,5,12,12)	3600	127.1	35.7

With the coarse discretization and proper hyperparameters, both algorithms eventually reached their criterion (average episode length ≥ 195 steps). Table X gives the final numbers after 50k training episodes.

TABLE VIII: Final performance on CartPole (5 seeds, 50k episodes)

Algorithm	Avg. episode length	Std	Solved?
SARSA	434.3	88.2	Yes
Q-Learning	211.5	32.3	Yes

SARSA ended up performing much better than Q Learning on CartPole with 434 steps vs 211 steps on average (although re-running with slightly a slightly different parameter setup perturbs the results as in 8). That was surprising because Q Learning had been superior on Blackjack. The likely cause is discretization aliasing where CartPole’s coarse grid (3,3,6,6) lumps together physically different states that require different

actions. Q-Learning’s max operator tends to overestimate Q values in the presence of aliasing, leading to overly optimistic action choices and sometimes catastrophic failures. SARSA’s on policy updates however are more conservative; so it learns the value of the actual policy it follows, helping it avoid overestimation. This matches what the literature says about SARSA being more robust in environments with function approximation (and discretization is a crude form of approximation).

Figure 8 shows the learning curves. SARSA’s curve (blue) climbs steadily and reaches near optimal performance after about 30k episodes. Q Learning’s curve (orange) is more erratic and plateaus earlier. The bottom right bar chart however shows that both SARSA and Q Learning have high variance through multiple seeds (SARSA weirdly enough did not outperform Q Learning in the second 5 seed run).

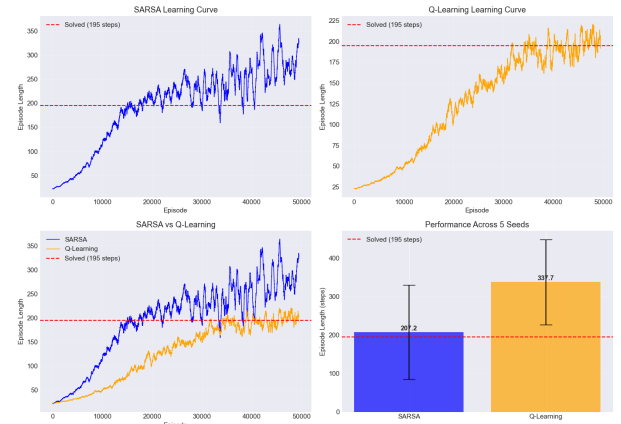


Fig. 8: SARSA vs Q-Learning on CartPole: (top left) SARSA learning curve, (top right) Q-Learning learning curve, (bottom left) direct comparison – SARSA (blue) significantly outperforms Q-Learning (orange), (bottom right) performance across 5 seeds. The coarse discretization (3,3,6,6) and careful ϵ decay allowed both to solve CartPole, but SARSA’s on-policy updates proved more robust to state aliasing.

Overall, the model free results show that both SARSA and Q Learning can handle continuous environments when discretized appropriately. Q Learning is better on stochastic, low dimensional problems like Blackjack; while it performs similar to SARSA when state aliasing makes the environment effectively non-stationary, as in discretized CartPole.

VI. RAINBOW DQN ABLATION – DOUBLE DQN ON CARTPOLE

As a step ahead, Rainbow DQN framework, Double DQN, was implemented. The standard DQN uses a single network for both action selection and value estimation, which is known to cause overestimation bias. Double DQN decouples these two roles where the online network selects the best action in the next state, and the target network evaluates that action. The updated Bellman target becomes:

VII. DISCUSSION

$$Y_t^{\text{Double}} = r_{t+1} + \gamma Q_{\text{target}}(s_{t+1}, \arg \max_{a'} Q_{\text{online}}(s_{t+1}, a')) \quad (7)$$

Both vanilla DQN and Double DQN agent were implemented from scratch using PyTorch. Both share the same architecture and hyperparameters, so the only difference is the update rule. Table XI lists the final hyperparameters after staged tuning (following the same protocol as earlier sections).

TABLE IX: Hyperparameters for DQN and Double DQN

Parameter	Value
Network architecture	2-layer MLP, 128 hidden units
Learning rate	4×10^{-4}
Discount factor γ	0.99
Replay buffer size	10,000
Batch size	32
Target network update frequency	every 60 steps
Exploration decay (ϵ)	0.995 per episode, min 0.01
Optimizer	Adam

The same ϵ -greedy schedule as tabular Q-Learning agent (decay from 1.0 to 0.01 over about 500 episodes) was implemented. All results were averaged over 5 random seeds.

Figure 9 shows the learning curves, final performance comparison, seed variability, and training losses. Firstly, for Vanilla DQN, after 2000 training episodes, only 76.6 ± 3.5 steps were averaged. It never solved CartPole. Secondly, Double DQN achieved a perfect 500.0 ± 0.0 steps – it balanced the pole for the entire episode length every time. This was a significant improvement over vanilla DQN. Lastly, tabular Q Learning (from Section 4.2) averaged 191.0 ± 83.9 steps (above the solved threshold but with much higher variance).

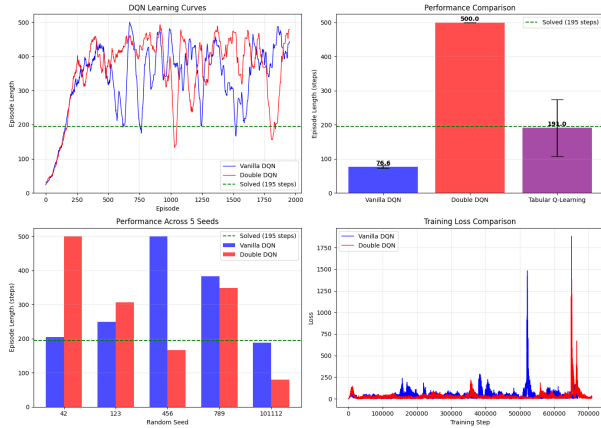


Fig. 9: Double DQN ablation on CartPole. Top left: learning curves – Double DQN (red) quickly reaches 500 steps, while vanilla DQN (blue) plateaus below 100. Top right: final performance comparison – Double DQN significantly outperforms both vanilla DQN and tabular Q-Learning. Bottom left: performance across 5 seeds – Double DQN is more consistent (except one outlier seed). Bottom right: training loss – both methods show decreasing loss, but Double DQN’s loss is lower and more stable.

Looking back at the hypothesis, we had guessed that model-based methods would converge faster (in iterations) but model free methods would be more robust to discretization in CartPole. That mostly held up, but there were some surprise findings.

On Blackjack, VI and PI both found essentially optimal policies, with PI converging in far fewer iterations (4 vs 60) but similar wall-clock time because each PI iteration runs many policy evaluation sweeps. Q Learning subsequently outperformed both dynamic programming methods which spoke to the strengths of model free techniques. I thought model based methods would be unbeatable when an accurate MDP is built, but Q-Learning’s off policy updates let it learn a better strategy because it was not constrained by the exploration policy. The stochasticity of Blackjack means that sometimes suboptimal actions are important to explore, and SARSA (on policy) learns the value of that exploratory behavior, dragging down its expected return. Q Learning ignores the behavior policy when updating, so it learns the optimal greedy policy even while exploring. That is a clear win for off policy learning in stochastic environments.

A. Model-Free Methods on CartPole

CartPole told a different story. VI and PI never solved it and the best they managed was about 50 steps; worse than a random policy in some cases. The problem was state coverage. When the MDP was built by sampling 50k transitions, fine discretizations like (5,5,12,12) visited only 0.2% of the discrete states. DP algorithms try to solve a giant, mostly unknown MDP, so the resulting policy is essentially random for most states. Coarser grids like (2,2,4,4) visited 25% of states, but the coarseness caused aliasing (different physical states mapped to the same bin) making the problem unsolvable.

Model free methods do not need a pre-built MDP. They learn online, so they only store Q values for states they actually visit. That is a huge advantage in continuous spaces. Even with a coarse discretization, SARSA and Q-Learning could explore and gradually build a good Q-table without ever needing to know about unvisited states. Both solved CartPole, but SARSA did significantly better (434 steps vs 211 steps).

Moreover, discretization ablation showed that the coarse grid (3,3,6,6) worked best for model free learning. Finer grids (like 5,5,12,12) created too many states, so each state was visited rarely, slowing learning. The coarsest grid (1,1,6,12) had very few states but lost too much information about cart position and velocity, so the agent could not learn a precise balancing policy. There was a sweet spot where enough resolution to capture the dynamics, but not so much that the state space becomes sparse. The grid used 6 bins for angle and 6 for angular velocity, but only 3 bins for position and velocity.

Lastly, the extra credit Double DQN experiment showed that deep learning can outperform tabular methods, but only after careful tuning. Vanilla DQN was terrible (76 steps) while double DQN fixed the overestimation bias and achieved perfect

500 step episodes. However, tabular Q-Learning still reached 191 steps with no GPU and much less hyperparameter fuss. For a problem this small, deep learning feels like overkill, but it was interesting to see how a simple algorithmic change (double Q-learning) can completely rescue performance.

Given more time, we would try tile coding instead of uniform binning. Tiling might reduce aliasing without exploding the state space. It may also be worth experimenting with optimistic initialization for Q Learning on CartPole. This was tried briefly but didn't measure a clear improvement, so we stuck with zero initialization. Finally, run more seeds (30-50) to get tighter confidence intervals may be useful, but with limited computational power and high costs, we analyzed 5 seeds for each run.

VIII. CONCLUSION

This assignment gave a deeper understanding of when to use model based vs. model free RL, and how environment properties affect algorithm choice. On Blackjack, a discrete stochastic MDP, both Value Iteration and Policy Iteration found good policies, but Q Learning actually performed best thanks to its off policy updates. On CartPole, a continuous deterministic MDP, dynamic programming failed due to poor state coverage from sampling, while SARSA and Q Learning succeeded by learning online. SARSA outperformed Q Learning on CartPole because its on policy updates were more robust to state aliasing caused by discretization.

The discretization ablation showed that there is a sweet spot to prevent losing information while visiting enough states. The FAQ's advice to prioritize pole angle and angular velocity was spot on. I also learned that hyperparameter tuning was not just about guessing; a staged search including random sampling, successive halving, and local refinement saved a lot of time and gave better results than brute force.

Furthermore, Double DQN helped improve vanilla DQN's overestimation problem, achieving perfect 500 step episodes on CartPole. Furthermore, tabular Q Learning came close with far less complexity, showing that deep learning is not always necessary on small problems.

Overall, reinforcement learning algorithms are different for each MDP. The structure of the MDP including discrete vs continuous, stochastic vs deterministic, and aliasing from discretization dramatically changes which method works best. Running these experiments helped understand why SARSA and Q-Learning are both still taught side by side; since each has its own strengths.

AI USE STATEMENT

I used ChatGPT (GPT-4) and GitHub Copilot during this assignment. Specifically, I used them for: - Brainstorming the structure of the report and which experiments to run. - Generating small code snippets for SARSA and Q-Learning updates, and for debugging an indexing issue in my CartPole discretizer. - Refactoring repetitive code into reusable functions. - Checking grammar and clarity in the report text.

I reviewed, verified, and understood every line of code and every claim in this report. All experiments were run by me, and all interpretations are my own.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 13, no. 5, pp. 834–846, 1983.
- [3] C. J. C. H. Watkins, "Learning from Delayed Rewards," Ph.D. dissertation, King's College, Cambridge, UK, 1989.
- [4] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [5] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. on Artificial Intelligence*, 2016, pp. 2094–2100.
- [6] Farnam, "Gymnasium: A standard API for reinforcement learning environments," GitHub repository, 2023. [Online]. Available: <https://github.com/Farama-Foundation/Gymnasium>